

Ch 1: Introduction to Basic Data Structure and Algorithm Design

Data Structure and Algorithm Design

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal
Former Guest Faculty, Darmstadt University of Applied Sciences, Germany

CamScanner



Overview

Ch 1: Introduction to Basic Data Structure and Algorithm Design

- Basic Data Structures
  - Arrays, Stack, Queue, Linked list, Tree and Graph
- Algorithm Design Techniques
  - Divide and Conquer
  - Greedy Algorithm
  - Backtracking Algorithms
  - Branch & Bound
  - Randomized Algorithms
- Algorithm Analysis
  - Time and Space Complexity
  - Asymptotic notations: Big O, Big Theta and Big Omega

CamScanner

3.Algorithm Analysis:

3.1.Best, Worst and Average Case Analysis
3.2.Rate of Growth
3.3.Asymptotic Notations- Big Oh, Big Omega and Big Theta

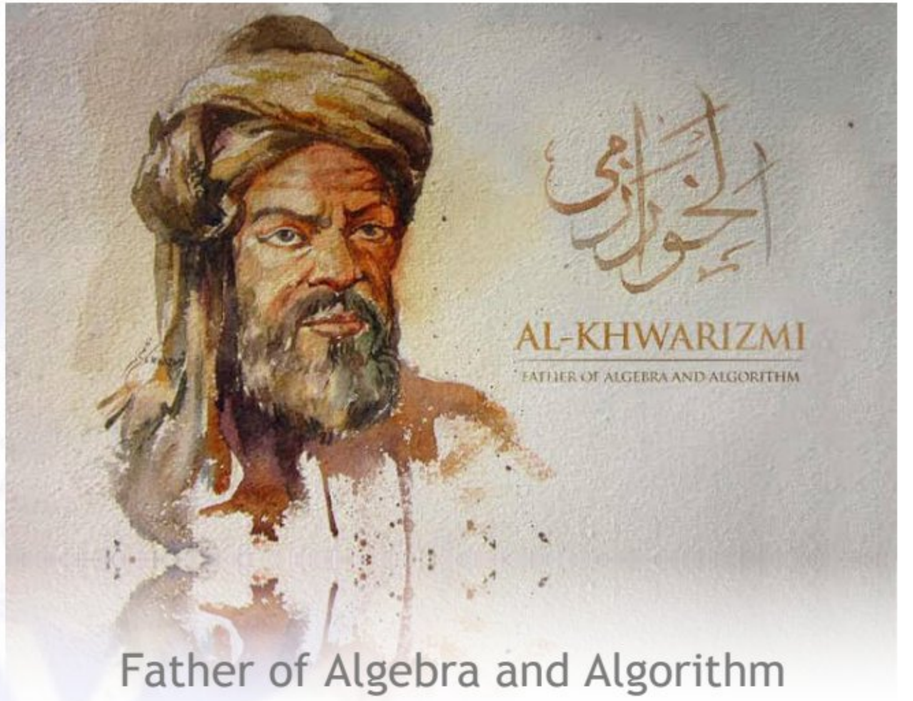
ALGORITHM ANALYSIS

CamScanner

Origin of word "algorithm"

What is and algorithm?

- The word "**algorithm**" came from the name of a Persian mathematician Abu Ja'far Mohammed ibn Musa al-KhowArizmi (c. 825 A.D.).
- His contributions to mathematics, geography, astronomy, and cartography established the basis for innovation in algebra and trigonometry.
- On the ***Calculation with Hindu Numerals***, written about 820, was principally responsible for spreading the **Hindu-Arabic numeral system** throughout the Middle East and Europe.



Father of Algebra and Algorithm

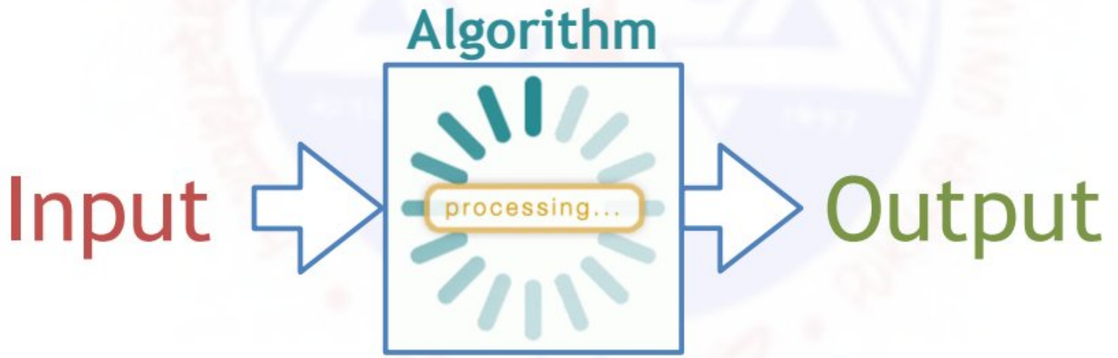
https://www.amust.com.au/2018/01/al-khwarizmi-from-algebra-to-computing/

- It was translated into Latin as **Algori**tmi de numero Indorum. Al-Khwārizmī, rendered as (Latin) **Algoritmi**, led to the term "**algorithm**".

CamScanner

What is and algorithm?

- The word “algorithm” has taken on a special significance in computer science where algorithm has come to refer to a method that can be used by a computer for the solution of a problem.
- An algorithm is a finite set of instructions that performs a particular task.
- Mathematically, an algorithm is a method consisting of a sequence of steps (that maps input to output.) that transform an input to the corresponding output.



- Algorithms are systematic methods for solving problems. The methods that you learn at school for multiplying or dividing integers are algorithms.

CamScanner

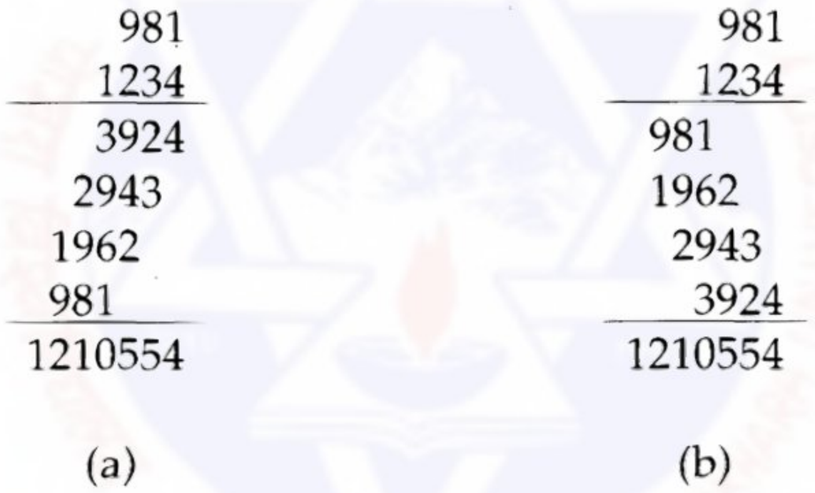
Characteristics of Algorithms

- All algorithm must satisfy the following criteria:
    1. **Input**: Zero or more quantities are externally supplied.
    2. **Output**: At least one quantity is produced.
    3. **Definiteness**: Each instruction in algorithm must be clear and unambiguous.
    4. **Finiteness**: An algorithm must terminate after a finite number of steps.
    5. **Concrete steps:** An algorithm must contain a series of concrete steps. Concrete means that the action described by a step in algorithm is completely understood and doable.
    6. **Correctness**: An algorithm must be correct and must produce desired output for a given input.
    7. **Feasible**: An algorithm must be **feasible** in terms of technical point of view or in terms of cost.
    8. **Language independent:** An algorithm must be **language independent** and its steps can be implemented using the code of any programming languages.

CamScanner

Efficiency of Algorithms

- Suppose you have to multiply two positive integers 981 and 1234 using only pencil and paper.



981	981
1234	1234
3924	981
2943	1962
1962	2943
981	3924
1210554	1210554
(a)	(b)

Figure 1.1. Multiplication (a) American (b) English

CamScanner

Efficiency of Algorithms

- Suppose you have to multiply two positive integers 981 and 1234 using only pencil and paper.

- Write the multiplicand and the multiplier side by side.
- Make two columns, one under each operand, by repeating the following rule until the number in the left-hand column is 1:
  - divide the number in the left-hand column by 2, ignoring any fractions, and
  - double the number in the right-hand column by adding it to itself.
  - Next cross out each row where the number in the left-hand column is even, and
  - finally add up the numbers that remain in the right-hand column.
- This algorithm, sometimes called multiplication *a la russe*, resembles the one used in the hardware of a binary computer.

981	1234	1234
490	2468	
245	4936	4936
122	9872	
61	19744	19744
30	39488	
15	78976	78976
7	157952	157952
3	315904	315904
1	631808	631808
		1210554

Figure 1.2. Multiplication *à la russe*

CamScanner

Efficiency of Algorithms

- Suppose you have to multiply two positive integers 981 and 1234 using only pencil and paper.

- For this algorithm, however, we require that both the multiplicand and the multiplier have the same number of figures, and furthermore that this number be a power of 2, such as 1, 2, 4, 8, 16, etc. This is easily fixed by adding zeros on the left if necessary: in our example, we add just one 0 on the left of the multiplicand, making it into 0981, so that both operands have four figures. Now to multiply 0981 by 1234 we first multiply the left half of the multiplicand (09) by the left half of the multiplier (12), and write the result (108) shifted left as many places as there are figures in the multiplier: four, in our example. Next we multiply the left half of the multiplicand (09) by the right half of the multiplier (34), and write the result (306) shifted left by half as many places as there are figures in the multiplier: two, in this case. Thirdly we multiply the right half of the multiplicand (81) by the left half of the multiplier (12), and write the result (972) also shifted left by half as many places as there are figures in the multiplier; and fourthly we multiply the right half of the multiplicand (81) by the right half of the multiplier (34) and write the result (2754), not shifted at all. Finally we add up the four intermediate results as shown in Figure 1.3 to obtain the answer 1210554.

	Multiply		Shift	Result
i)	09	12	4	108····
ii)	09	34	2	306··
iii)	81	12	2	972··
iv)	81	34	0	2754
				1210554

Figure 1.3. Multiplying 0981 by 1234 by divide-and-conquer

CamScanner

Efficiency of Algorithms

- When we have a problem to solve, **we can design several algorithms** to solve it.
- Among several algorithms we can choose the best (i.e. **most efficient**).
- The **most efficient algorithm** is the algorithm that uses the **minimum resources** (execution time, memory space, etc.) to perform a task.
- Some algorithms may take less time to obtain the result and others may take less memory space during its execution
- The **advantage of having multiple solutions** to a problem is that **you can choose** an efficient algorithm **based on your preference** whether you need an algorithm that takes **less time** or **less memory**.
- Another important point is that **an algorithm A might be more efficient than algorithm B** for a specific variation of the given problem or for specific class of inputs.
    - For example, one sorting algorithm might be efficient for sorting small number of integers or another might be more efficient to sort large number of integers.



CamScanner

Efficiency of Algorithms

- Having multiple solutions (algorithms) to a problem raises the question of how to decide which of the several algorithm is best or most efficient (preferable).
- **Analysis of an algorithm**
  - refers to the task of determining how much resources (computing time, storing space etc.) the algorithm requires.
  - is the process of determining computational complexity of algorithms.
- The **computational complexity** of an algorithm
  - is the amount of resources required for the algorithm to produce its output.
  - the resources required for an algorithms includes
    - the computing time the algorithm takes to produce an output (**time complexity**),
    - the space required for the algorithm (**space complexity**) and
    - the other resources like internet, communication band etc. it may need for its processing.
- Two approaches for determining efficiency of algorithms:
  - Empirical approach
  - Theoretical approach

CamScanner

Efficiency of Algorithms

- **Empirical Approach:**
  - this approach consists of programming the competing algorithms, trying then on different instances of input with the help of a computer and choosing the one which is fast in execution and uses minimum space.
- **Theoretical Approach:**
  - This approach consists of determining mathematically the quantity of resources needed by each algorithm as a function of the size of the instances considered.



CamScanner

Empirical Approach

- The empirical comparison works as below:
  - Develop the algorithms using some programming languages.
  - Execute those algorithms in some computers.
  - Provide a suitable range of inputs.
  - Measure how much resources the algorithms use to produce the output.
  - Choose the algorithm which uses minimum resources.
- Is this approach a better solution for comparing the algorithms? Can you figure out the weakness of this approach?



CamScanner

Empirical Approach

- **Factors affecting Empirical Approach:**
  - The following are some factors that may influence a fair comparison of algorithms:
    - The effort involved in the programming and testing the two algorithms.
    - How efficiently the coding of the two programs that use the algorithms are written. One of the program may be better written than other.
    - The choice of test case or input data set may unfairly favor one of the two algorithm.
    - Efficiency of the operating system used.
    - The specifications of the hardware used.



CamScanner

Empirical Approach

- **Factors affecting Empirical Approach:**
  - Empirical approach depends on the various factors like the programming language and compiler's quality you used to implement the algorithm, the speed of the CPU, bus and other peripheral hardwares, the quality of the operating systems that you are using, the hardware specification of the computer where you are running the algorithms, other programs that are currently running at your system or on your networks during the comparison, coding efficiency of programmer who convert the algorithm into a program etc.
  - Therefore, you have to develop the two algorithms with the same effort with same programming quality and under the same conditions, you have to compare them by compiling using the same complier on the same computer.
  - This approach does not work if you want to compare the two algorithms which are developed in two different environments.



CamScanner

Theoretical Approach

- The advantages of the theoretical approach
  - it depends on neither the computer being used (machine independent), nor the programming language, nor the skill of the programmer.
- These all can be achieved in theoretical approach by using **asymptotic analysis of algorithms** or simply **asymptotic analysis** to measure their efficiencies.
  - **Asymptotic analysis** attempts to estimate the resource consumption of an algorithm.
  - **Asymptotic analysis** allows us to compare the **relative costs** of two or more algorithms for solving the same problem.



CamScanner

Input Size

- One primary factor that need to be consider to analysis the performance of an algorithm is size of input
- The input size is the number of data items that need to be processed.
- Suppose you are given a list of integer numbers to sort in ascending order.
  - The number of data items in the given list is the input size.
  - As the input size increases, the time taken by the algorithm to sort the list grows



CamScanner

Running Time

- The running time of an algorithm depends on the size of input.
- The running time of the algorithm is defined as the number of primitive operations or steps executed in the algorithm.
- Different line in the algorithm may take different time for execution. However, we assume each line of algorithm or pseudocode require a constant amount of time to execute.



CamScanner

Example 1: Finding a largest number in a array.

Running Time

- Let us consider an algorithm to find the largest element in an array of size n

```
Algorithm Largest(A, n):
 Input: An array A with n integer elements.
 Output: The largest element in A.
currentLarge ← A[0]
for i ← 1 to n −1 do
 if  currentLarge < A[i] then
    currentLarge ← A[i]
return currentLarge
```

Fig. 2.1: Pseudo-code of an algorithm that finds the largest element (integer) in an array

CamScanner

Example 1: Finding a largest number in a array.

Running Time

- Suppose you are given a task to find the total cost for running time of the algorithm **`Largest(A, n)`**.
- What you have to do is you need to find the time cost of each statement in the algorithm and the number of times each statement of the algorithm are executed.

	Algorithm Largest(A, n)	**Cost**	**Times**
1	*currentLarge* ← A[0]	C1	1
2	**for** i ← 1 **to** n−1 **do**	C2	n
3	**if** *currentLarge* < A[i] **then**	C3	n-1
4	*currentLarge* ← A[i]	C4	n-1
5	**return** *currentLarge*	C5	1

Fig. 2.2: Cost calculation of algorithm **`Largest(A, n)`**



CamScanner

Example 1: Finding a largest number in a array.

Running Time

- Suppose you are given a task to find the total cost for running time of the algorithm `Largest(A, n)`.
- What you have to do is you need to find the time cost of each statement in the algorithm and the number of times each statement of the algorithm are executed.

	Algorithm Largest(A, n)	**Cost**	**Times**
1	*currentLarge* ← A[0]	C1	1
2	**for** i ← 1 **to** n-1 **do**	C2	n
3	**if** *currentLarge* < A[i] **then**	C3	n-1
4	*currentLarge* ← A[i]	C4	n-1
5	**return** *currentLarge*	C5	1

- The total running time $T(n)$ of the algorithm is given by

$$T(n) = c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5$$



CamScanner

Example 1: Finding a largest number in a array.

Running Time

$$T(n) = c_1 + c_2n + c_3(n-1) + c_4(n-1) + c_5$$

- The value of $T(n)$ depends on the position of the largest number in the given array.
- Suppose the elements in the array are sorted in descending order.
- The first element in the array is the largest element.
- If the first element of the array is largest, then the statement 3 is always false and statement 4 is never executed.
- This is called **best-case analysis** of the algorithm and it is given by

$$T(n) = c_1 + c_2n + c_3(n-1) + c_4 * 0 + c_5$$
$$= (c_2 + c_3)n + (c_1 - c_3 + c_5)$$

- This can be expressed in the form of $an + b$ where $a = c_2 + c_3$ and $b = c_1 - c_3 + c_5$.
- Therefore, $T(n)$ of the given algorithm in **best case** is a linear function of n.



CamScanner

Example 1: Finding a largest number in a array.

Running Time

$$T(n) = c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5$$

- Suppose the elements in the array are sorted in ascending order.
- This means last element in the array is the largest element.
- If the last element of the array is largest, then the statement 3 is always true and statement 4 is always executed. That is statement 4 is executed for $(n-1)$ times.
- This is called **worst-case analysis** of the algorithm. Therefore, $T(n)$ in worst-case is given by

$$\begin{aligned} T(n) &= c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5 \\ &= (c_2 + c_3 + c_4)n + (c_1 - c_3 - c_4 + c_5) \end{aligned}$$

- This can also be expressed in the form of $an + b$ where $a = c_2 + c_3 + c_4$ and $b = c_1 - c_3 - c_4 + c_5$.
- Therefore, $T(n)$ of the given algorithm in worst case is also a linear function of n.



CamScanner

Example 2: Insertion Sort

Running Time

	Algorithm INSERTION-SORT(A, n)	**Cost**	**Times**
1	**for** $j \leftarrow 1$ **to** $n-1$ **do**	c_1	n
2	temp ← A[j]	c_2	$n-1$
3	▷Insert A[j] into the sorted array	0	0
4	i ← j-1	c_4	$n-1$
5	**while** i>=0 **and** temp <= A[i] **do**	c_5	$\sum_{j=1}^{n-1} t_j$
6	A[i+1] ← A[i]	c_6	$\sum_{j=1}^{n-1} (t_j - 1)$
7	i ← i - 1	c_7	$\sum_{j=1}^{n-1} (t_j - 1)$
8	A[i+1] ← temp	c_8	$n-1$



CamScanner

Running Time

- The total running time $T(n)$ of the INSERTION-SORT(A, n) algorithm is given by

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=1}^{n-1} t_j +$$
$$c_6 \sum_{j=1}^{n-1} (t_j - 1) + c_7 \sum_{j=1}^{n-1} (t_j - 1) + c_8(n-1)$$



CamScanner

Running Time

- **Best-case of INSERTION-SORT(A,n):** Suppose the elements in the given array are already sorted. In such case, `temp <= A[I]` is always false. Therefore, the sixth and seventh never executed i.e. the number of time, the sixth and seventh statements execute is zero. However, the fifth statement executes only for one time for a particular value of $j = 1, 2, \ldots, n-1$. Therefore, $T(n)$ for best-case is given by
- $T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_6 * 0 + c_7 * 0 + c_8(n-1)$
- $= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$
- $= c_1 n + c_2 n - c_2 + c_4 n - c_4 + c_5 n - c_5 + c_8 n - c_8$
- $= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$
- $= an + b$ where $a = c_1 + c_2 + c_4 + c_5 + c_8$ and $b = -(c_2 + c_4 + c_5 + c_8)$.
- Therefore, $T(n)$ of the given INSERTION-SORT(A,n) algorithm in best-case is a linear function of n.



CamScanner

Running Time

- **Worst-case of INSERTION-SORT(A, n) :** Suppose the elements in the given array are already reversed order i.e. the array is already sorted in descending order. In such case, `temp <= A[I]` is always true. If you trace the algorithm, you will find that for $j = 1$, the sixth and seventh statements executed for 1 times, for $j = 2$, they execute for 2 times, for $j = 3$, they execute for 3 times and for $j = n - 1$, they execute for $n - 1$ times. Therefore, the number of times, the sixth and seventh statements execute is equal to
- $= 1 + 2 + 3 + \ldots + n - 1$
- $= (1 + 2 + 3 + \ldots + (n - 1) + n) - n$
- $= \frac{n(n+1)}{2} - n$
- $= \frac{n(n-1)}{2}$
- The fifth statement i.e. the header of the *while*-loop executes one more time than that of the sixth and seventh statements. For $j = 1$, the fifth statement executed for 2 times, for $j = 2$, it executes for 3 times, for $j = 3$, it executes for 4 times and finally for $j = n - 1$, it executes for n times. Therefore, the number of times, the fifth statement executes is equal to
- $= 2 + 3 + \ldots + n - 1 + n$
- $= (1 + 2 + 3 + \ldots + n - 1 + n) - 1$
- $= \frac{n(n+1)}{2} - 1$



CamScanner

Running Time

- Therefore, $T(n)$ of the given `INSERTION-SORT(A,n)` algorithm for worst-case is given by
- $T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(\frac{n(n+1)}{2} - 1) + c_6(\frac{n(n-1)}{2}) + c_7(\frac{n(n-1)}{2}) + c_8(n-1)$
- $= (\frac{c_5 + c_6 + c_7}{2})n^2 + (c_1 + c_2 + c_4 + c_8 - \frac{c_6 + c_7 - c_5}{2})n - (c_2 + c_4 + c_5 + c_8)$
- This worst-case running time $T(n)$ can be expressed in the form of $an^2 + bn + c$ where $a = \frac{c_5 + c_6 + c_7}{2}$

  $b = c_1 + c_2 + c_4 + c_8 - \frac{c_6 + c_7 - c_5}{2}$ and $c = -(c_2 + c_4 + c_5 + c_8)$
- Therefore, $T(n)$ of the given `INSERTION-SORT(A,n)` algorithm in worst-case is a quadratic function of n.



CamScanner

Best, worst and average case analysis

- You have just studied what are the best-case and worst-case analysis with the example of INSERTION-SORT(A, n) algorithm.
- For a sorting algorithm, best-case occurs if the input array is already sorted.
- In contrast, the worst-case occurs if the input array is in reverse order i.e. the element of the array are sorted in descending order.
- We normally use the worst-case analysis for determining the running time of an algorithm since it gives the longest time that the algorithm may take for the worst scenario of input set.
- The running time doesn't go beyond the worst-case running time.



CamScanner

Best, worst and average case analysis

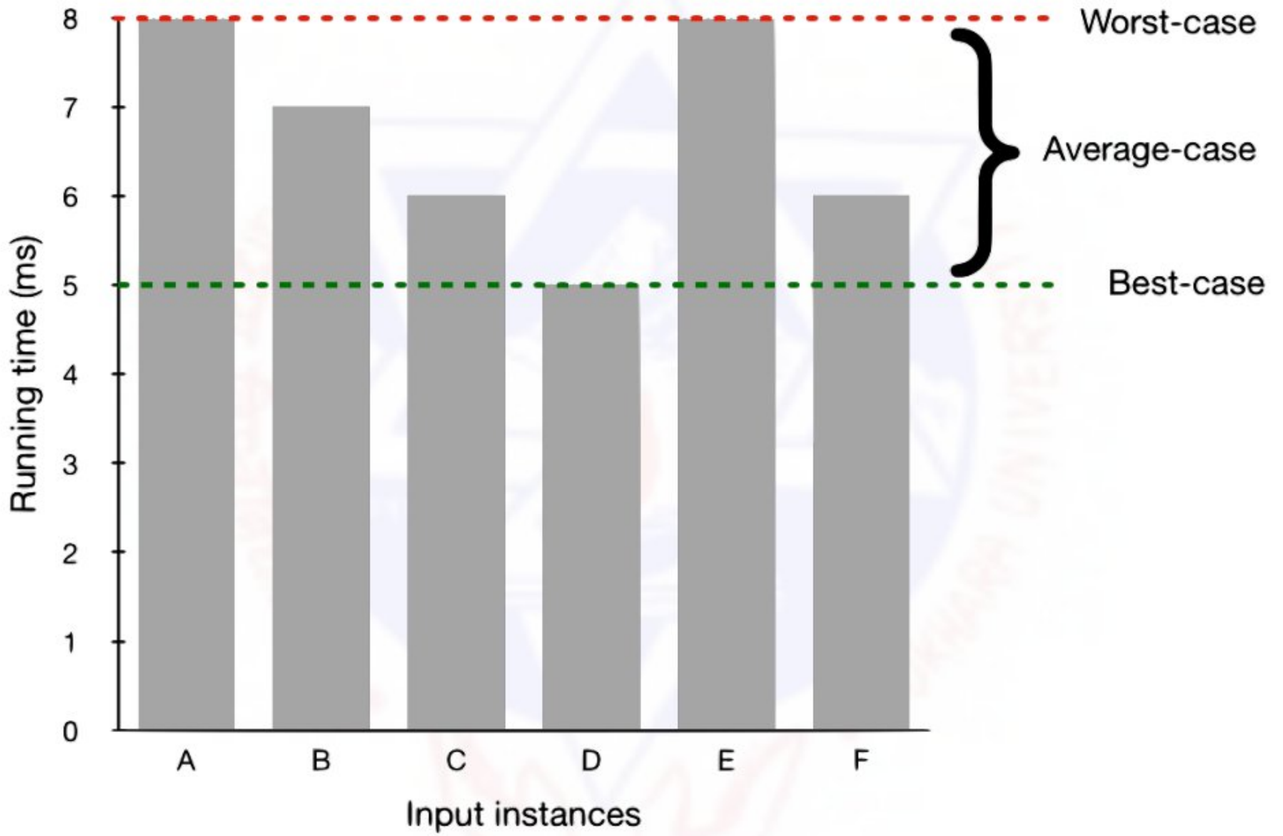


Fig. 2.3 : Running time of an algorithm in different instances of input set

CamScanner

Best, worst and average case analysis

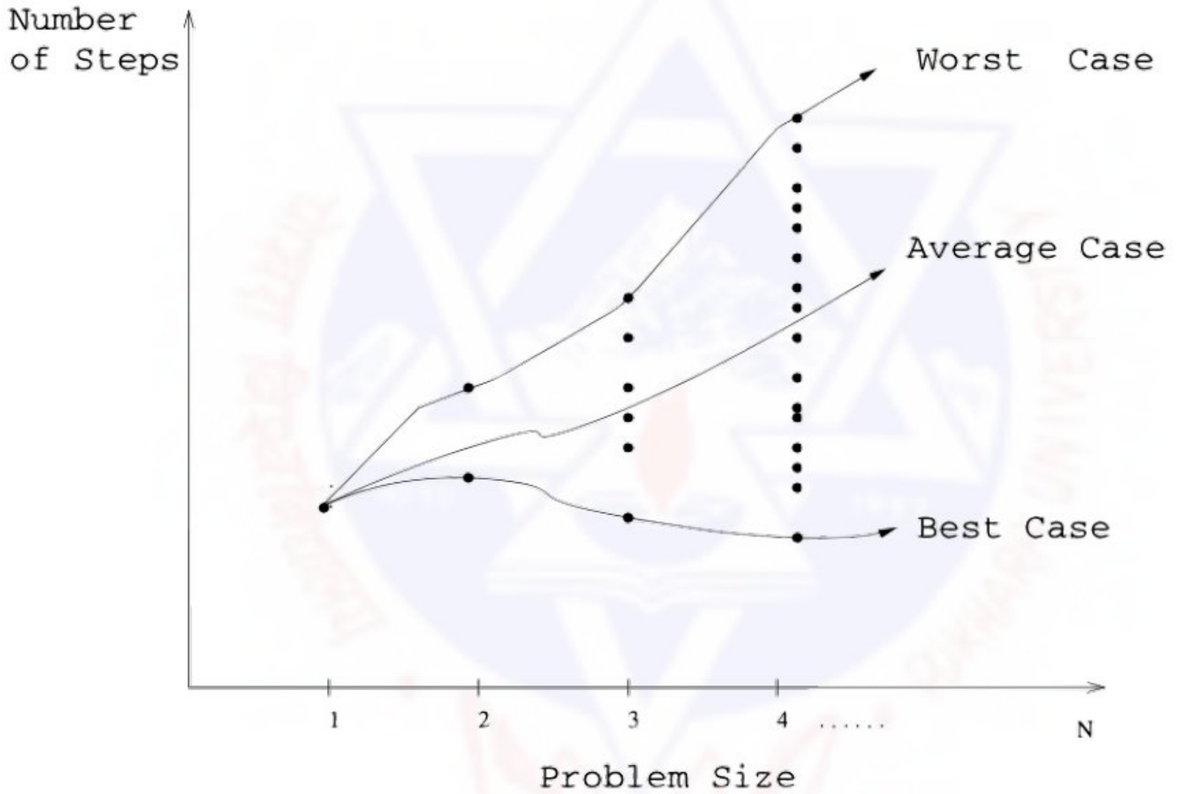


Figure 2.1: Best, worst, and average-case complexity

CamScanner

Best, worst and average case analysis

- The **worst-case** complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n. This represents the curve passing through the highest point in each column.

- The **best-case** complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n. This represents the curve passing through the lowest point of each column.

- The **average-case** complexity of the algorithm, which is the function defined by the average number of steps over all instances of size n.

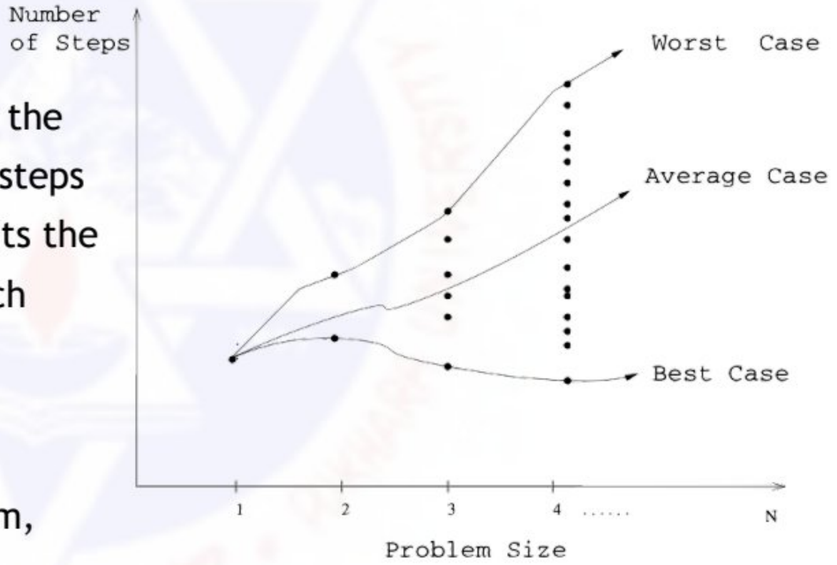


Figure 2.1: Best, worst, and average-case complexity

CamScanner

Best, worst and average case analysis

- Average-case running time is roughy as bad as the worst case running time.
- There is no way to exactly define what the average-input for a problem.
- We can use a randomized algorithm to randomly choose n numbers so that half of the elements are assumed to be in the sorted form if it is the case of sorting algorithm.
- The average-case running time is roughly the half of the worst-case running time.
- $$T(n)_{avg-case} = \frac{T(n)_{worst-case}}{2} = \frac{an^2 + bn + c}{2} = \frac{a}{2}n^2 + \frac{b}{2}n + \frac{c}{2}$$
- Which is again turned into a quadratic function of input size of n.
- Therefore, we normally use the worst-case running time to determine the performance of an algorithm.



CamScanner

Rate of Growth

- The amount of resource an algorithm consumes/demands depends on the amount of the input size.
- If you increase the input size, the running time (space required) may also increase.
- As we go increasing its input size, the amount of resources required for the algorithm keep on growing.
- The rate of growth (or order of growth or growth rate) is defined as the rate at which the resource consumption (i.e. time or space or any other resource requirement) of an algorithm increases as the input size grows.



CamScanner

Rate of Growth

- In our previous examples, you saw the running time of algorithm can be a **linear** or **quadratic** function of the input size.

Table 2.2: Some common growth rate functions

SN	Function of n	Name of Growth Rate	Example
1	c	Constant Growth Rate	Stack's Push or Pop
2	$log_b(n)$	Logarithmic (for base b) Growth	Binary Search
3	n	Linear Growth Rate	Sequential search
4	$nlog_b(n)$	Linear logarithmic Growth Rate	Quick sort
5	n^2	Quadratic Growth Rate	Insertion Sort
6	n^3	Cubic Growth Rate	Matrix Multiplication
7	2^n	Exponential Growth Rate	Tower of Hanoi



CamScanner

Growth Rates & Amount of Resource Demand

- You should know how much more resource your algorithm will demand when the amount of input size is increased.

Table 2.3: Amount of running time of different algorithms with different growth rates for varying input size n = 10, 20, 30, 40, 60, 80 and 100

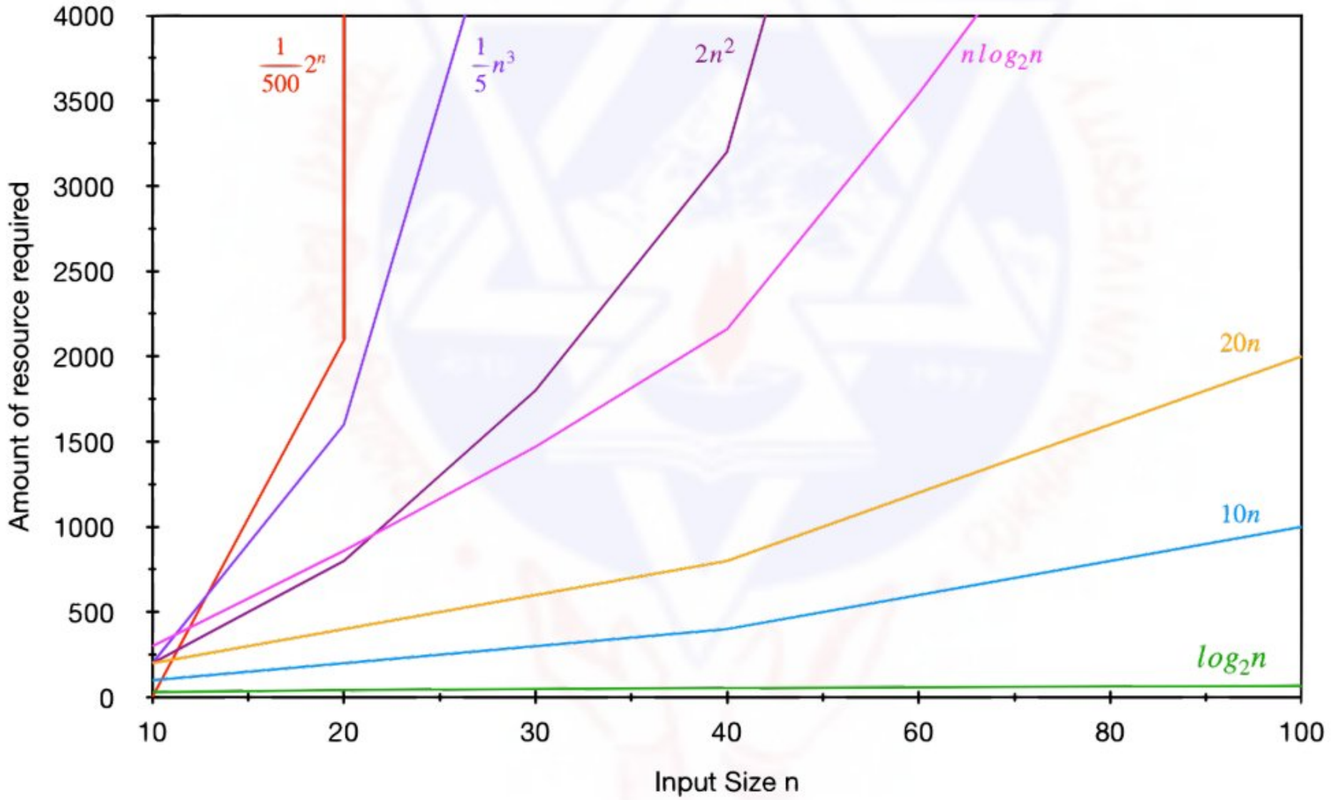
SN	Functions with common growth rates	Input Size (n)						
		10	20	30	40	60	80	100
1	Logarithmic ($10\,log_2 n$)	30	43	49	54	59	63	66
2	Linear function ($10n$)	100	200	300	400	600	800	1000
3	Linear function ($20n$)	200	400	600	800	1200	1600	2000
4	Linear Logarithmic ($n\,log_2 n$)	300	860	1470	2160	3540	5056	6640
5	Quadratic function ($2n^2$)	200	800	1800	3200	7200	12800	20000
6	Cubic function ($\frac{1}{5}n^3$)	200	1600	5400	12800	43200	102400	200000
7	Exponential function ($\frac{1}{500}2^n$)	2	2097	2.1E+06	2.2E+09	2.3E+15	2.4E+21	2.5E+27



CamScanner

Growth Rates & Amount of Resource Demand

- The Fig. 2.5 shows graphical visualization of the same information in the Table 2.3.



CamScanner

Growth Rates & Amount of Resource Demand

- The Fig. 2.5 shows graphical visualization of the same information in the Table 2.3.
- Here, you can easily understand how the amount of resource required by algorithm (with different growth rates) grows as the input size increases.
- The logarithmic ($log_2 n$) growth rate shows demanding of very small amount of resource (e.g. running time) even in a very large grow in the input size.
- The linear growth rate grows more quickly than that of the logarithmic growth rate. However, it grows much slowly than that of the quadratic growth rate, cubic growth rate and exponential growth rate.



CamScanner

Growth Rates & Amount of Resource Demand

- If the growth rate of running time of an algorithm is more slower, the algorithm is more efficient.
- It means the algorithm demands very small amount of resource even in the large increase in the input size.
- Suppose you have two algorithms- A and B. If the algorithm A has the growth rate of quadratic function and algorithm B has the growth rate of linear function, then the algorithm B is more efficient than A. Suppose you have another algorithm C whose growth rate is of logarithmic function, then the algorithm C is the most efficient.



CamScanner

Comparison of Algorithms using Growth Rate

- Suppose you are given a task to analyze two algorithms- algorithm A which has the running time $T_A(n) = 100n + 2$ and algorithm B with running time $T_B(n) = n^2 + n + 2$.

Table 2.6: Comparing growth rates of Algorithms A and B

Input size n	Running time of Algorithm A	Running time of Algorithm B
5	502	32
10	1002	112
50	5002	2552
99	9902	9902
100	10002	10102
500	50002	250502
1000	100002	1001002
10000	1000002	100010002



CamScanner

Comparison of Algorithms using Growth Rate

- For $n = 5$, running time of A is approx. 15 times longer than that of the B.
- For $n = 10$, it is approx. 9 times longer than that of the B.
- For $n = 99$, the running times of both algorithms are same.
- After $n = 99$, the algorithm A needs less running time even for the large n.
- For $n = 500$, 1000 and 10000, the rate of growth of running time of algorithm A is much slower than that of the algorithm B.
- Therefore, **the algorithm A is much better than the algorithm B.**
- The function whose leading term is n^2 grows faster than the function whose leading term is n.
- The running time of B is better than A for small value of n (i.e. $n < 99$) because the leading term of algorithm B has a large coefficient (i.e. 100).

CamScanner

Assumption for Finding Growth Rate

- Let us take the example of the **insertion-sort** algorithm.
- In best-case, $T(n) = an + b \approx n$
- In worst-case, $T(n) = an^2 + bn + c \approx n^2$ where a, b and c are some constants.
- Both the multiplicative coefficients and the lower-order terms of $T(n)$ are relatively insignificant for large enough input n.
- Therefore, we only take the leading term of $T(n)$ ignoring all the lower-order terms and constant coefficient.
- You can see in Table 2.6 where the algorithm A has large coefficient (100) in the leading term. Due to this, the algorithm appears to be inefficient until $n < 99$ but when $n > 99$, the algorithm A beats the algorithm B.



CamScanner

Assumption for Finding Growth Rate

- To determine the order of growth, we have ignored the insignificant terms and coefficients.
- That is we are not finding the exact order of growth. It means we are determining the **asymptotic efficient** of algorithm.
- In other words, we are doing **asymptotic analysis of algorithms**.
- In the case of algorithms presented in Table 2.6, algorithm A is asymptotically more efficient that the algorithm B.
- Note that an algorithm with lower order of growth in worst-case is more efficient than another.
- Beware of very large constant factors. An algorithm running in time 1,000,000 n is still O(n) but might be less efficient on your data set than one running in time $2n^2$, which is O(n^2)



CamScanner

Asymptotic Analysis

- To compare the efficiency of algorithms, it is not often important to find their exact running time or the exact memory space they need.
- It is sufficient to find the rate of growth of a resource that the algorithms demands as their input size increases.
- To find only the rate of growth of resources, the lower-order terms and coefficients in $T(n)$ can be discarded by only considering the leading term as $n \to \infty$.
- These rate of growths of algorithms are then compared to choose an efficient algorithm.
- This is called the **asymptotic analysis of algorithm.**
- A function $g(n) = n^3 + 2n^2 - 7$ behaves asymptotically like $n^3$and
- A function $h(n) = nlogn + 5n$ behaves asymptotically like $nlogn$.



CamScanner

Asymptotic Analysis

- The asymptotic analysis of algorithms uses standard notations called ***asymptotic notations***.
- The following are the commonly used asymptotic notations to describe the running time of algorithms:
    1. Big Oh Notation (O)
    2. Big Omega Notation (Ω)
    3. Big Theta Notation (Θ)
- ***Upper bound:*** The upper bound for growth of $T(n)$ shows highest growth rate that an algorithm can have
- ***Lower bound:*** it is the lowest growth rate that an algorithm can have.



CamScanner

Asymptotic Analysis

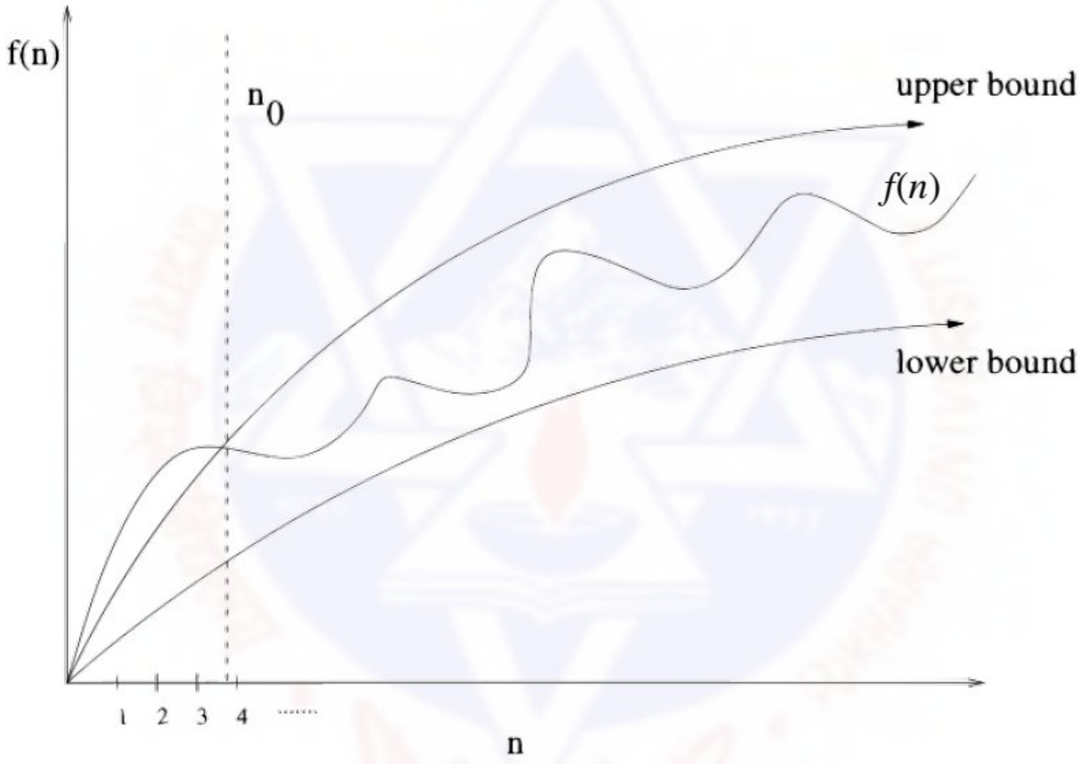


Figure 2.2: ***Upper*** and ***lower*** bounds valid for $n > n_0$ smooth out the behavior of complex functions

The Big Oh Notation (O)

- The big Oh notation (O) describes the highest growth rate (i.e. the maximum possible amount) of a resource that an algorithm demands for the input size $n \to \infty$.
- The big Oh notation (O) represents the ***asymptotic upper bound*** of a resource that an algorithm demands. That is it describes the growth rate for the worst-case of an algorithm.
- For example, the worst-case running time of insertion sort is $O(n^2)$.
- Read this $O(n^2)$ as "***in order of square n***".



CamScanner

The Big Oh Notation (O)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n for upper bound.
- Suppose a positive constant c is a coefficient of the upper bound function $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big Oh notation to represent the upper bound of $T(n)$, written as $O(g(n))$ (read as big oh of g of n), is then mathematically defined as:

  $O(g(n)) = \{T(n)$: there exists positive constants c and n_0 such that $0 \leq T(n) \leq cg(n)$ for all $n \geq n_0\}$.

  i.e. $T(n) = O(g(n))$ (We have used '=' instead of '$\in$' to indicate $T(n) \in O(g(n))$)

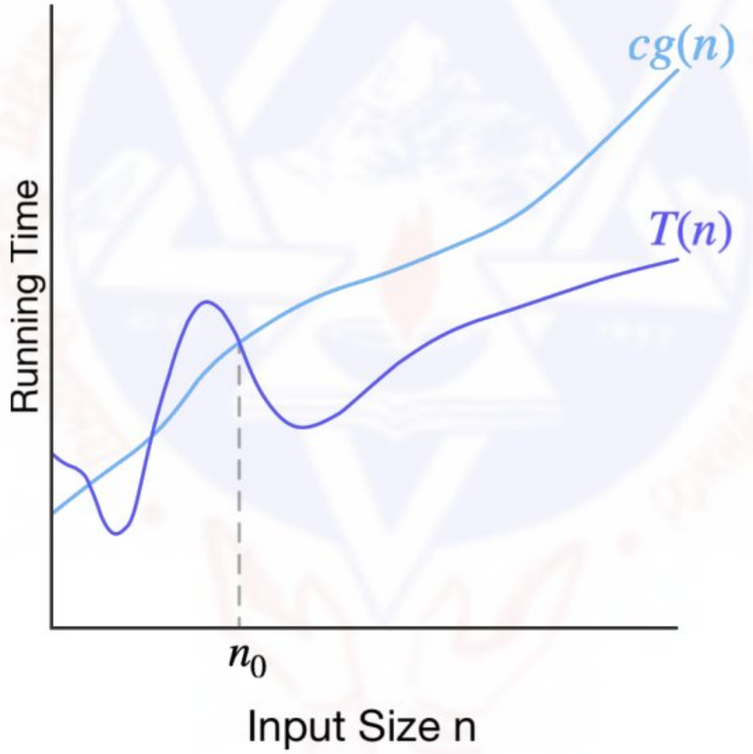


CamScanner

The Big Oh Notation (O)

The big Oh notation to represent upper bound of $T(n)$, written as $O(g(n))$ is defined as:

$O(g(n))$ = {$T(n)$: **there exists positive constants** c **and** n_0 **such that** $0 \leq T(n) \leq cg(n)$ **for all** $n \geq n_0$**}.**



The Big Oh Notation (O)

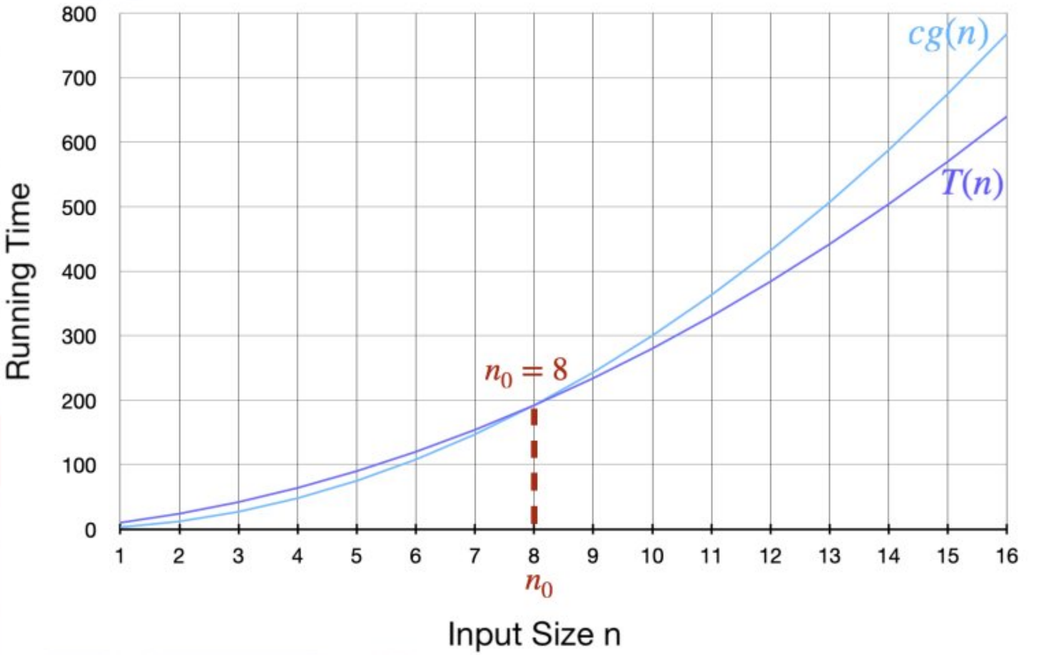
- **Example:** To illustrate how the big oh notation (O) is used to represent the upper bound of running time of an algorithm,
  - let us consider the running time function of an algorithm as $T(n) = 2n^2 + 8n$.
  - Suppose the upper bound function $g(n) = n^2$ (The $g(n)$ is nothing other than the leading term in $T(n)$ ignoring its coefficient) and a positive constant $c = 4$.
  - The values of $T(n)$ and $cg(n)$ for n = 1, 2, 3, … , 15 are shown in Table 2,7.



CamScanner

The Big Oh Notation (O)

n	$T(n) = 2n^2 + 8n$	$cg(n) = 4n^2$	Remarks
1	10	4	
2	24	12	
3	42	27	
4	64	48	
5	90	75	
6	120	108	
7	154	147	
8	192	192	$0 \leq T(n) \leq cg(n)$ and $(n_0 = 8)$
9	234	243	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
10	280	300	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
11	330	363	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
12	384	432	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
13	442	507	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
14	504	588	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
15	570	675	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$



The time complexity is given by

$$T(n) = O(g(n))$$
$$= O(n^2) \quad \text{since } g(n) = n^2.$$

CamScanner

The Big Omega Notation (Ω)

- The big omega Ω notation describes the least possible amount of the running time $T(n)$ of an algorithm that it needs for a large enough input size (i.e. $n \to \infty$).
- Since the big omega notation describes the asymptotic lower bound of the running time for an algorithm, it is used to describe the best case of an algorithm.
- For example, the best-case running time of insertion sort is $\Omega(n)$.



CamScanner

The Big Omega Notation (Ω)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n to describe the lower bound.
- Suppose a positive constant c is a coefficient of the lower bound function $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big omega notation to represent the lower bound of $T(n)$, written as $\Omega(g(n))$ (read as big omega of g of n), is then mathematically defined as:
  - $\Omega(g(n)) = \{T(n)$: there exists positive constants $c$and $n_0$such that $0 \leq cg(n) \leq T(n)$ for all $n \geq n_0\}$.
  - i.e. $T(n) = \Omega(g(n))$ (We have used '=' instead of '$\in$' to indicate $T(n) \in \Omega(g(n)))$

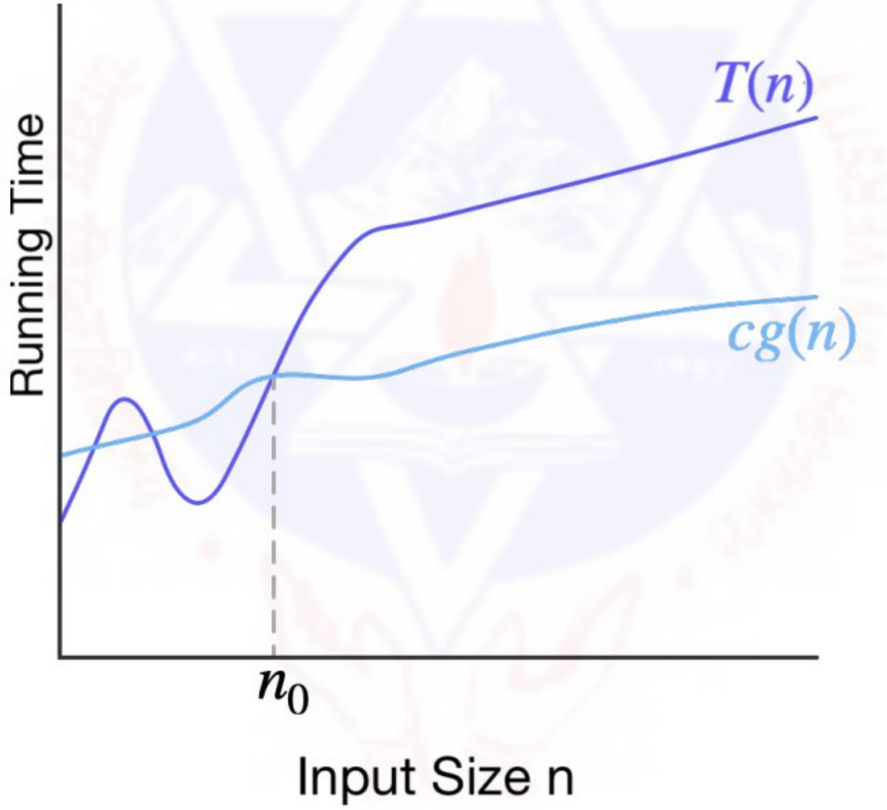


CamScanner

The Big Omega Notation (Ω)

The big omega notation to represent lower bound of $T(n)$, written as $\Omega(g(n))$, is defined as:

$\Omega(g(n))$ = {$T(n)$: **there exists positive constants** c **and** n_0**such that** $0 \leq cg(n) \leq T(n)$ **for all** $n \geq n_0$ **}.**



CamScanner

The Big Theta Notation (Θ)

- The big theta notation Θ describes both the asymptotic upper bound and lower bound of running time of algorithms.
- That is the big theta notation binds the running time function $T(n)$ of an algorithm from below and above.
- This notation is, therefore, generally used to describe the average-case of an algorithm.



CamScanner

The Big Theta Notation (Θ)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n to describe the upper and lower bound.
- Suppose a positive constants c_1 and c_2 are coefficients of $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big theta notation to represent the lower bound of $T(n)$, written as $\Theta(g(n))$ (read as big theta of g of n), is then mathematically defined as:

  $\Theta(g(n))$ = {$T(n)$: there exists positive constants c_1, c_2 and n_0 such that $0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n)$ for all $n \geq n_0$ }.

  i.e. $T(n) = \Theta(g(n))$ (We have used '=' instead of '$\in$' to indicate $T(n) \in \Theta(g(n))$)



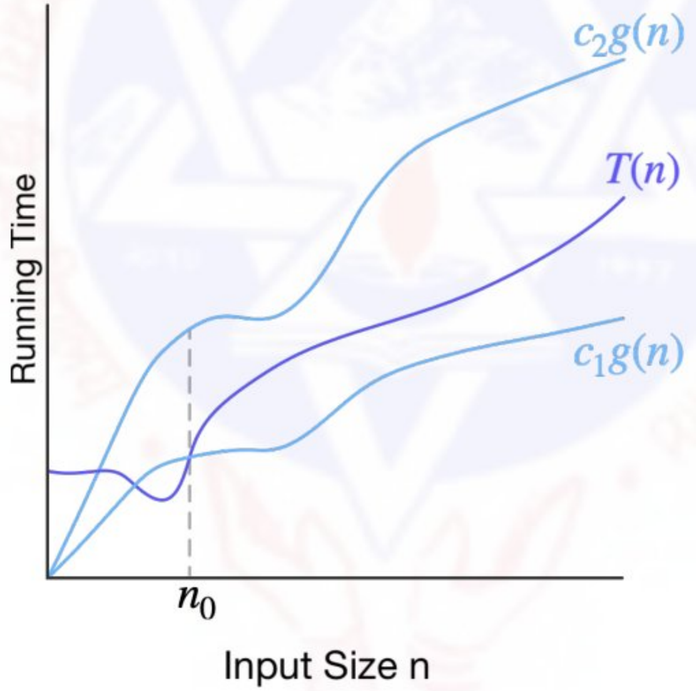
CamScanner

The Big Theta Notation (Θ)

The big theta notation $\Theta(g(n))$ to represent the lower bound of $T(n)$, is defined as:

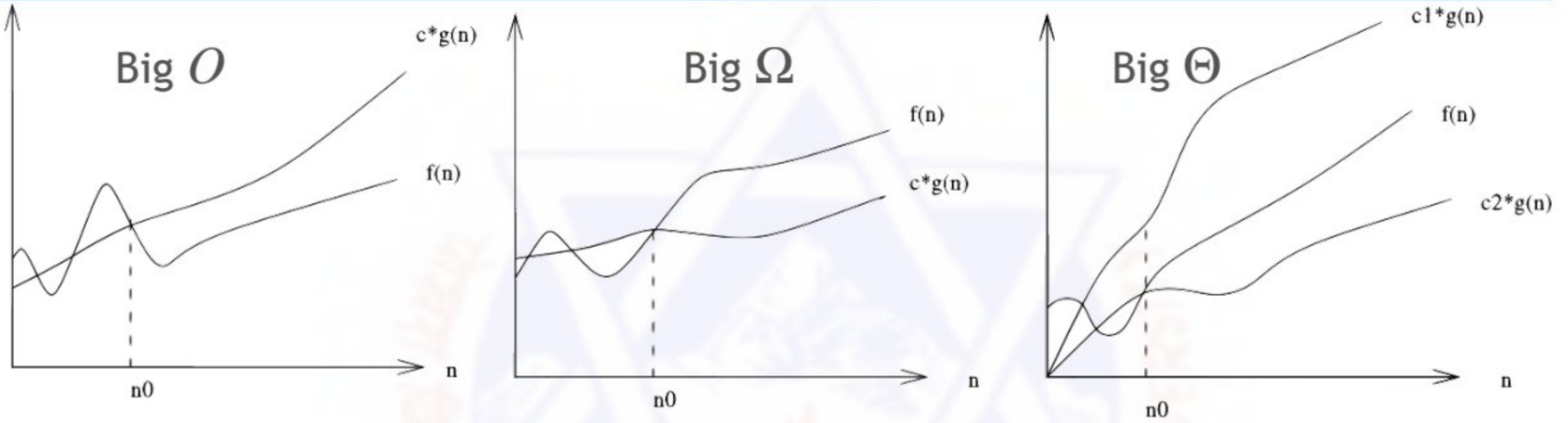
$\Theta(g(n))$ = {$T(n)$: there exists positive constants c_1, c_2 and n_0 such that

$$0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}.$$



CamScanner

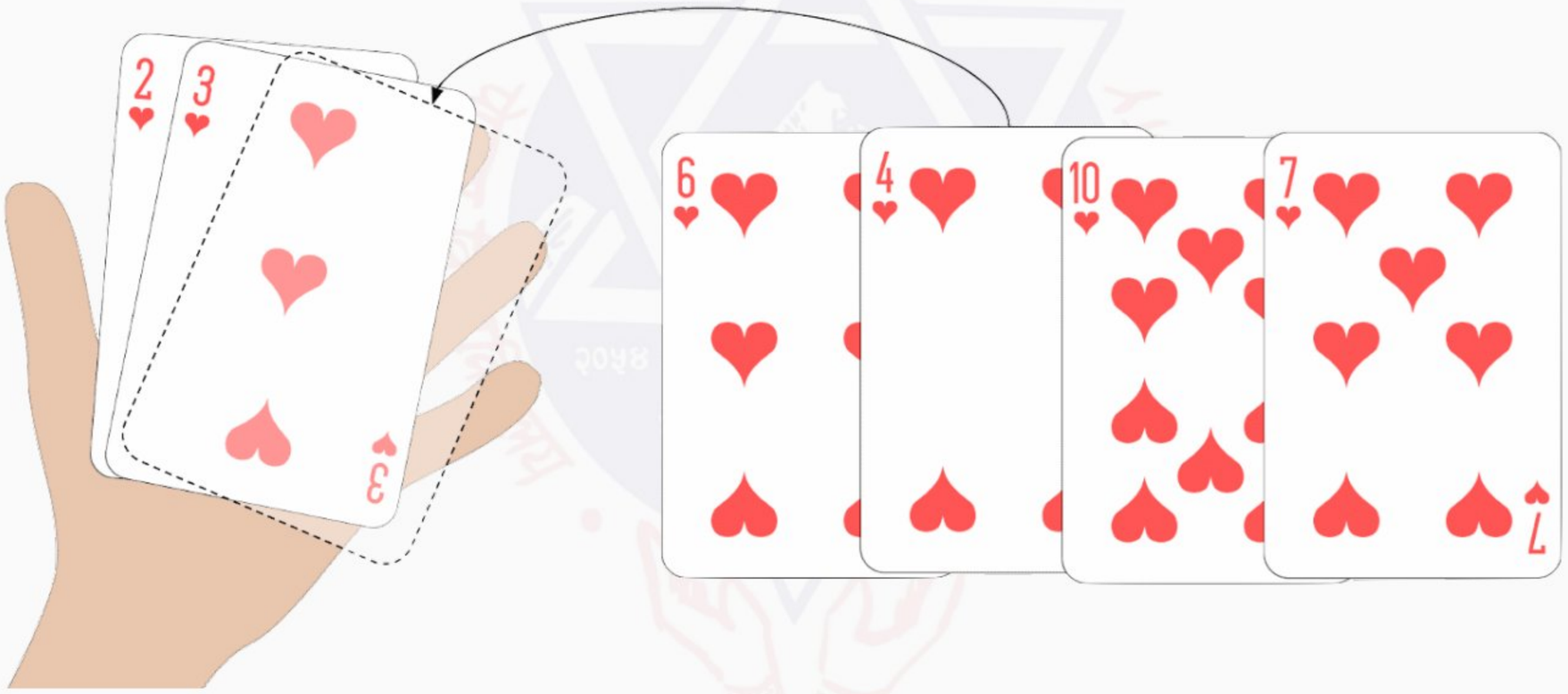
The Big Theta Notation (Θ)



- $f(n) = O(g(n))$ means $c \cdot g(n)$ is an *upper bound* on $f(n)$. Thus there exists some constant c such that $f(n)$ is always $\leq c \cdot g(n)$, for large enough n (i.e. , $n \geq n_0$ for some constant n_0).
- $f(n) = \Omega(g(n))$ means $c \cdot g(n)$ is a *lower bound* on $f(n)$. Thus there exists some constant c such that $f(n)$ is always $\geq c \cdot g(n)$, for all $n \geq n_0$.
- $f(n) = \Theta(g(n))$ means $c_1 \cdot g(n)$ is an upper bound on $f(n)$ and $c_2 \cdot g(n)$ is a lower bound on $f(n)$, for all $n \geq n_0$. Thus there exist constants c_1 and c_2 such that $f(n) \leq c_1 \cdot g(n)$ and $f(n) \geq c_2 \cdot g(n)$. This means that $g(n)$ provides a nice, tight bound on $f(n)$.

CamScanner

Selection Sort



CamScanner

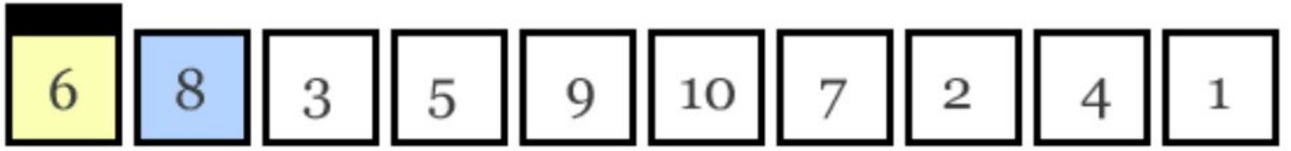
Selection Sort

- Following are the steps involved in selection sort(for sorting a given array in ascending order):
    1. Starting from the first element, we search the smallest element in the array, and replace it with the element in the first position.
    2. We then move on to the second position, and look for smallest element present in the subarray, starting from index 1, till the last index.
    3. We replace the element at the second position in the original array, or we can say at the first position in the subarray, with the second smallest element.
    4. This is repeated, until the array is completely sorted.



CamScanner

Selection Sort



Yellow is smallest number found
Blue is current item
Green is sorted list



CamScanner

Selection Sort

```
void selectionSort(int arr[], int n)
{
    int i, j, min_idx;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
        if (arr[j] < arr[min_idx])
            min_idx = j;

        //Swap the found minimum element with the first element
        swap(&arr[min_idx], &arr[i]);
    }
}
```



CamScanner

Analysis of Selection Sort

- Worst Case Time Complexity: $O(n^2)$
- Best Case Time Complexity: $O(n^2)$
- Average Time Complexity: $O(n^2)$

Class Discussion



CamScanner

Bubble Sort

- The bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will swap both the elements, and then move on to compare the second and the third element, and so on.
- If we have total n elements, then we need to repeat this process for n-1 times.
- It is known as bubble sort, because with every complete iteration the largest element in the given array, bubbles up towards the last place or the highest index, just like a water bubble rises up to the water surface.



CamScanner

Bubble Sort

```
// A function to implement bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n; i++)
    {
      // Last i elements are already in place
     for (j = 0; j < n-i-1; j++)
      {
        if (arr[j] > arr[j+1])
         {
          temp = arr[j];
          arr[j] = arr[j+1];
          arr[j+1] = temp;
         }
      }
    }
}
```



CamScanner

Analysis of Bubble Sort

- Worst Case Time Complexity: $O(n^2)$
- Best Case Time Complexity: $O(n)$
- Average Time Complexity: $O(n^2)$

Class Discussion



CamScanner



THANK YOU

CamScanner